INSTRUCTIONS

Please read the instructions carefully before doing the questions.

- You can use materials in your computer, notebook and text book.

- You are **NOT allowed** to use any device to share data with others.

Beside the above conditions, students must follow the following requirements:

1. The work must complete by using Visual Studio 2022++

2. The Framework must be .NET 8.0

3. THIS PART IS VERY IMPORTANT, PLEASE READ IT CAREFULLY AND FOLLOW THE INSTRUCTIONS.

- You are given a database script (.sql file) in Zip file. Execute the script before doing questions.
- **You must use the given solution.**
- **You are not allowed to add any more libraries via NuGet Package Manager into given solution.**
- **Submission Guideline:**

  Submit your work for each question separately. For each question, please:

  - Publish your project using the command:

    ```
    dotnet publish -c Release -o ./[QuestionNumber_StudentAccount]
    ```

    Example:

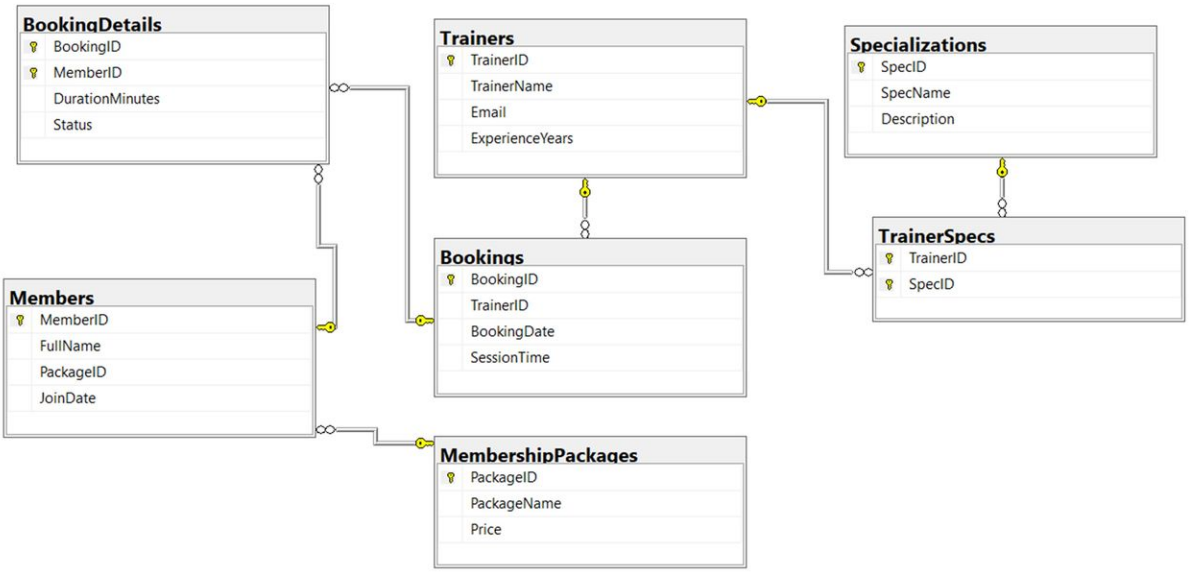
    ```
    dotnet publish -c Release -o ./Q1_trungnthe123432
    ```

  - Submit the root folder of the project into the PEA_Client application.
  - If the root folder of the project is too large, you may delete the following subfolders to reduce its size before submitting: /bin, /obj

Just one of above requirements is violated, your work will be considered as invalid.

Question 1:

You are provided with a database as following diagram



Ensure that the database connection string is adjustable through **appsettings.json** (otherwise your post will get 0 point) as pattern:

```
{
    "ConnectionStrings":
    {
        "MyCnn":""
    }
}
```

Your task is to implement the following APIs using **.NET 8** and **Entity Framework Core**:

Method	URL	Description
GET	`/api/Specializations`	Retrieve all fitness specializations with trainer count statistics. Supports JSON.
GET	`/api/filter-trainers`	Advanced search for trainers with booking duration stats and recent trainee trace logs.
POST	`/api/trainers`	Add a new gym trainer to the database.
DELETE	`/api/trainers/{trainerId}`	Remove a trainer and clear their spec links.

1. GET /api/Specializations

- Retrieve a list of all fitness specializations from the database.

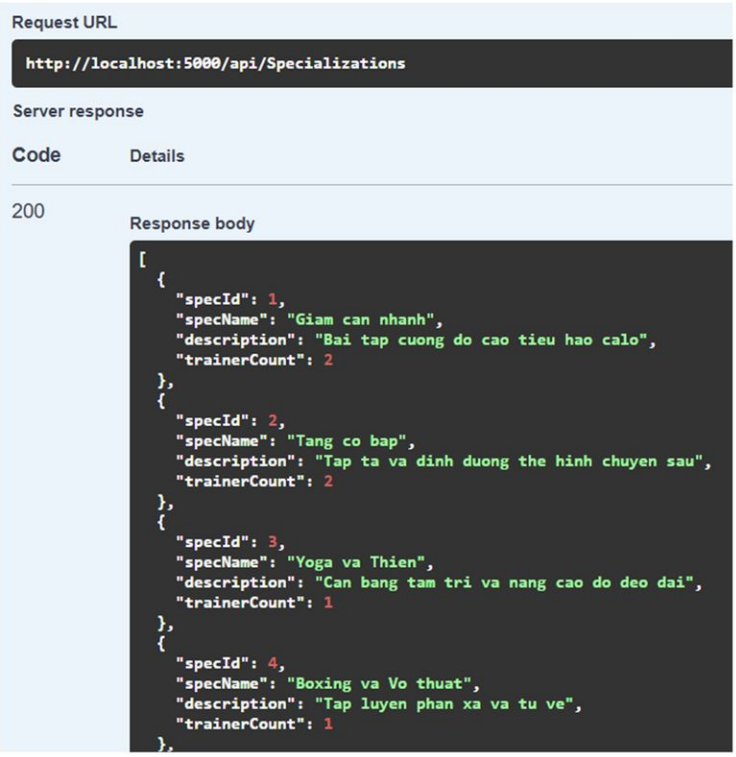
- Response Fields:

`SpecId` (int)

`SpecName` (string)

`Description` (string)

`TrainerCount` (int): Total number of trainers holding this specialization.



2. GET /api/filter-trainers

Retrieve a list of gym trainers with advanced booking performance statistics.

- Filtering Logic:

`specId` (int, optional): If specified, return only trainers associated with this Specialization ID.

`minExperience` (int, optional): If specified, return only trainers whose experience (years) is greater than or equal to this value.

- **Response Fields**:

`TrainerId`, `TrainerName`, `Email`, `ExperienceYears`.

`TotalMinutesTrained` (int): The total sum of `DurationMinutes` conducted by this trainer across all booking sessions in the `BookingDetails` table. Return **0** if there are no sessions.

`LastBookedMember` (string): The `FullName` of the gym member who booked a session with this trainer most recently (based on `Bookings.SessionTime` descending). If no sessions are booked, return "**No Bookings**".

`SpecializationList` (List of string): A collection of `SpecName` categories this trainer specializes in.

- **Error handling**: Return **400 Bad Request** if `minExperience` is negative (if provided).

- **Sorting**: Sort results by `ExperienceYears` in **descending** order.

Request URL

```
http://localhost:5000/api/filter-trainers?specId=1&minExperience=2
```

Server response

Code	Details
200	Response body


```
[
  {
    "trainerId": 1,
    "trainerName": "HLV Nguyen Hoan",
    "email": "hoan.nguyen@gym.com",
    "experienceYears": 5,
    "totalMinutesTrained": 60,
    "lastBookedMember": "Cao Van Nam",
    "specializationList": [
      "Giam can nhanh",
      "Tang co bap"
    ]
  },
  {
    "trainerId": 4,
    "trainerName": "HLV Pham Huy",
    "email": "huy.pham@gym.com",
    "experienceYears": 2,
    "totalMinutesTrained": 45,
    "lastBookedMember": "Phan Thanh Thao",
    "specializationList": [
      "Giam can nhanh"
    ]
  }
]
```

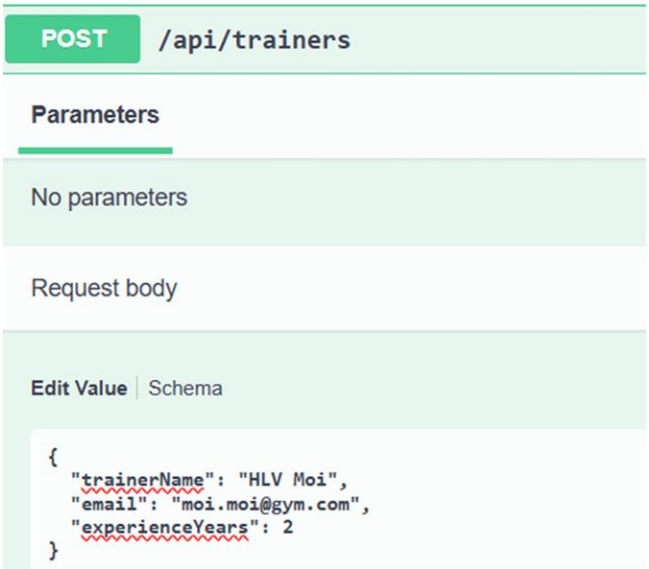


3. POST /api/trainers

Accept `TrainerName`, `Email`, and `ExperienceYears` inside the request body.

- **Return**: **201 Created** along with the complete newly created `Trainer` entity (ensure navigation properties like `Bookings` and `Specs` / `Specializations` are returned as empty lists `[]` instead of `null` to avoid client integration crashes).

- **Error handling**: If the trainer's `Email` already exists (case-insensitive check), return **400 Bad Request** with message `"Trainer email already exists"`.



Request URL

```
http://localhost:5000/api/trainers
```

Server response

Code	Details
201 *Undocumented*	Response body

```
{
  "trainerId": 7,
  "trainerName": "HLV Moi",
  "email": "moi.moi@gym.com",
  "experienceYears": 2,
  "bookings": [],
  "specs": []
}
```

POST `/api/trainers`

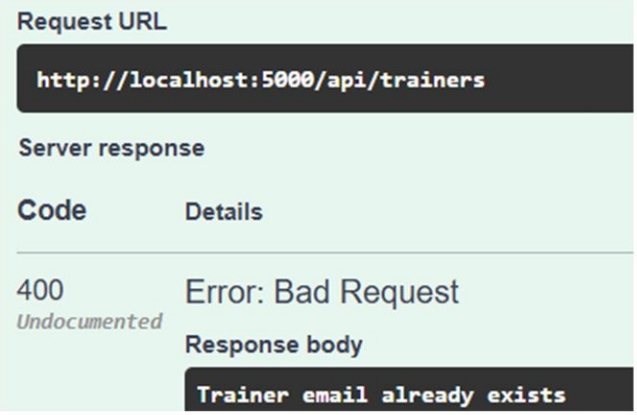
Parameters

No parameters

Request body

Edit Value | Schema

```
{
  "trainerName": "HLV Moi",
  "email": "hoan.nguyen@gym.com",
  "experienceYears": 2
}
```



4. DELETE /api/trainers/{trainerId}

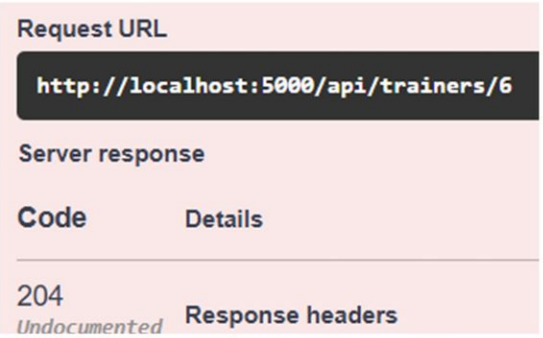
Delete the trainer with the specified `trainerId` and their related association records in the `TrainerSpecs` table.

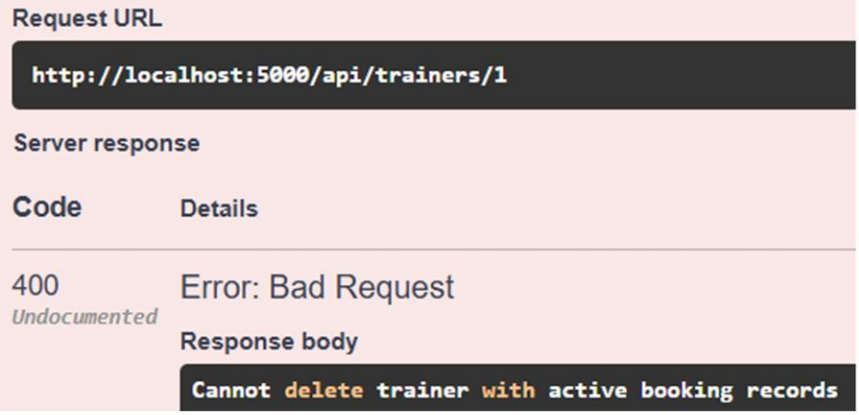
- Return: **204 No Content** if the operation is successful.

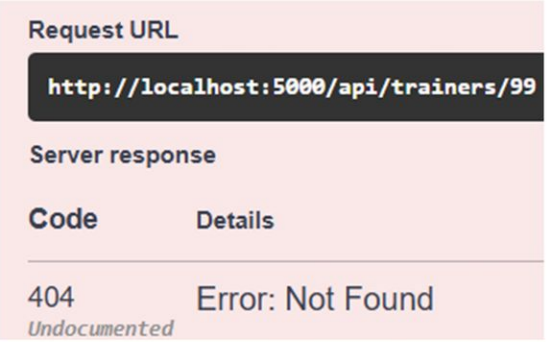
- Error handling:

If the trainer still has active sessions scheduled in the `Bookings` table, return **400 Bad Request** with message `"Cannot delete trainer with active booking records"`.

If the `trainerId` does not exist in the database, return a **404 Not Found** error.







Question 2:

In this question, you are asked to write an MVC/Razor Pages application. The application fetch data by calling pre-existing RESTful API hosted at GivenAPIBaseUrl. The API are provided in a separate project named GivenAPIs, which students must run locally to start the API server

1. Important Notes

- Students **MUST** use HttpClient to call the API.

- The GivenAPIBaseUrl must be written in appsettings.json

```
{
    "GivenAPIBaseUrl": "http://localhost:5100"
}
```

- Students must use **explicit string concatenation** for API endpoints (e.g., baseUrl + "/endpoint").

- All input and output elements in the HTML source **must** have an **'id'** attribute to ensure accessibility and traceability (Students can refer to the html files in given materials, which provides code snippets for the assignment).

2. Provided APIs

GET /api/booking-statuses : Returns a list of all distinct booking statuses (e.g., "Da tap", "Da huy", "Cho mien") as strings.

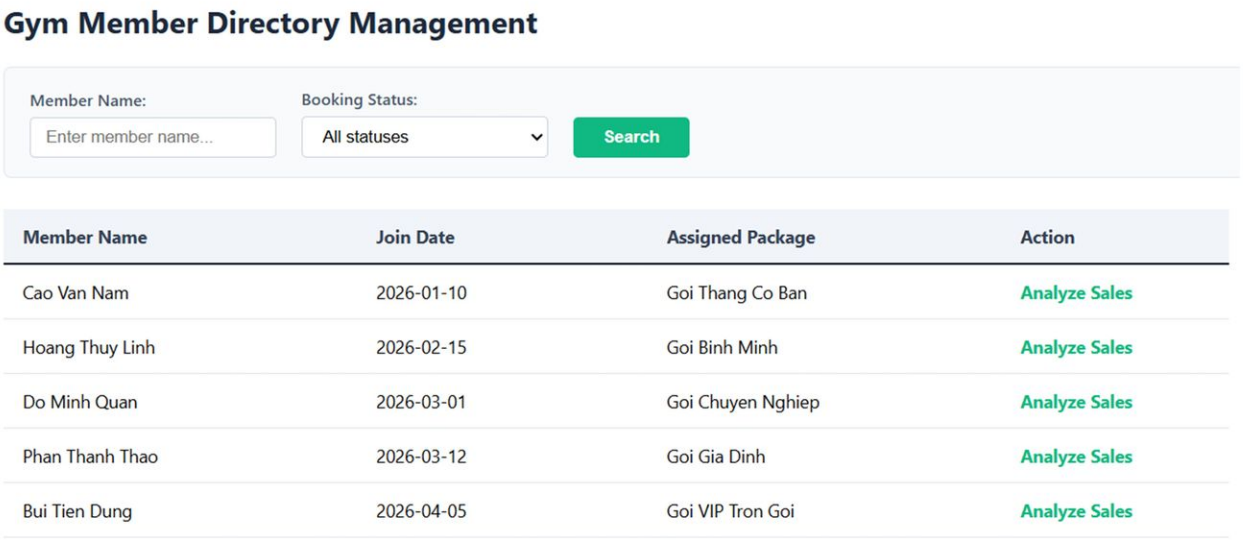
GET /api/members/search?memberName={name}&bookingStatus={status} : Returns a list of members filtered by partial name match and their training session status.

GET /api/members/{memberId} : Returns detailed information of a member, their current membership package, and a log of all their personal training sessions.

3. Requirements

3.1. Gym Member Directory Page

- URL : /Member



- Search Area :

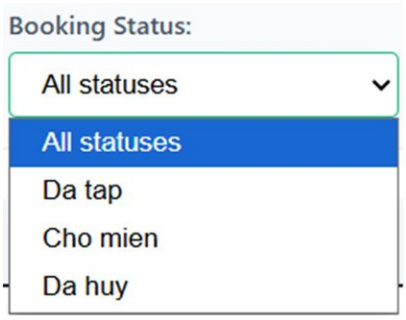
An **Input (Text)** for **Member Name** (id: **ip_memberName**) to search by member's full name or partial name.

A **Dropdown (Select)** for **Booking Status** (id: **sl_bookingStatus**).

The first option **"All statuses"** is set by **default**, must have id **op_all**, using the <option> tag.

Each specific status option must have id **op_{status}** (e.g., id="op_Da_tap").

A **Search** button (id: **bt_search**) to filter the table below.



- **Data Table** : Display matching gym members with columns:

Member Name : Display member's full name with id **td_name_{MemberID}**.

Join Date : Display registration date with id **td_joinDate_{MemberID}** (format: yyyy-MM-dd).

Assigned Package : Display registered package name with id **td_package_{MemberID}**.

Action : A link **"Analyze Sales"** (id: **a_{MemberID}**) redirecting to /Member/Analyze/{MemberID}.

3.2. Member Packages & Booking Intensity Page

- **URL** : /Member/Analyze/{MemberID}



Member Profile Analysis

Member ID: 1
Full Name: Cao Van Nam
Join Date: 2026-01-10
Current Package: Goi Thang Co Ban (300,000 VND)

Attended Sessions Log

Trainer Name	Session Time	Duration	Financial Value Tier
HLV Nguyen Manh	2026-06-26 09:00	60 mins	Standard Session

- Display basic information :

Member ID : Display member ID inside a with id **span_{MemberID}**.

Full Name : Display member's full name inside a with id **span_fullName**.

Join Date : Display join date inside a with id **span_joinDate** (format: yyyy-MM-dd).

- Display Attended Sessions Log : A table showing all scheduled personal training sessions of this member, including:

Trainer Name : Display assigned trainer name with id **td_trainer_{BookingID}**.

Session Time : Display session date-time with id **td_time_{BookingID}** (format: yyyy-MM-dd HH:mm).

Duration : Display session duration in minutes with id **td_duration_{BookingID}** (e.g., 90 mins).

Financial Value Tier :

Calculate: EstimatedCost = DurationMinutes * 5,000 (5,000 VND per trained minute).

If EstimatedCost >= 400,000: Display **"High Premium Session"** (Color: Red, Font-weight: bold).

Otherwise: Display **"Standard Session"** (Color: Green).

The cell must have id **td_tier_{BookingID}**.

4. Summary of HTML Elements ID (with Examples)

Page	Element	Tag	Id	Example
/Member	Input Member Name	<input>	ip_memberName	<input id="ip_memberName" type="text" />
	Dropdown Status	<select>	sl_bookingStatus	<select id="sl_bookingStatus">... </select>
	Status Option	<option>	op_{status}	<option id="op_Da_tap">Da tap</option>
	Button Search	<button>	bt_search	<button id="bt_search">Search</button>
	Cell: Member Name	<td>	td_name_{MemberID}	<td id="td_name_2">Hoang Thuy Linh</td>
	Analyze Sales Link	<a>	a_{MemberID}	Analyze Sales
/Member/Analyze/{MemberID}	Field MemberId		span_{MemberID}	2
	Cell: Trainer Name	<td>	td_trainer_{BookingID}	<td id="td_trainer_3">HLV Le Hai</td>

	Cell: Value Tier	<td>	td_tier_{BookingID}	<td id="td_tier_3">High Premium Session</td>

5. Summary of Required URLs

Function	**URL**
List members, filter by text input name and status dropdown	/Member
View member's package profile and estimated session costs	/Member/Analyze/{MemberID}